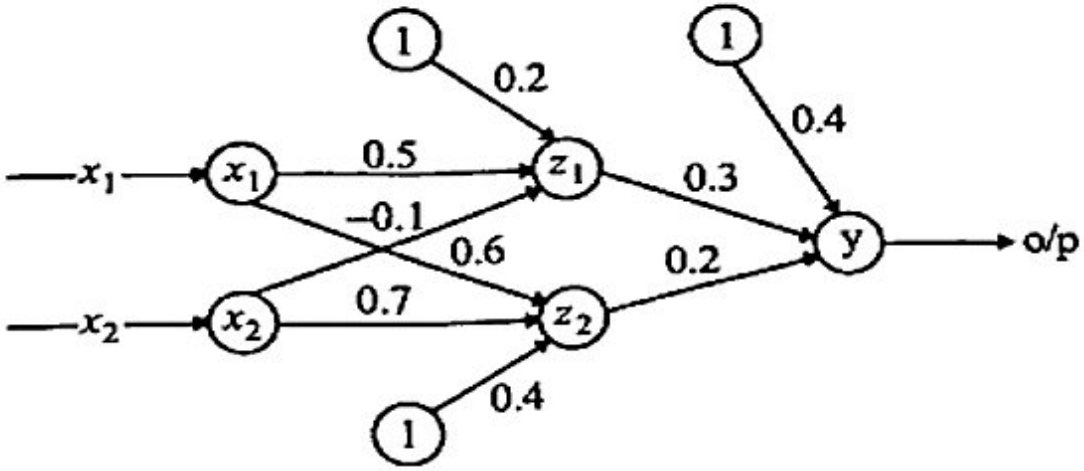
Question Bank Part-II
Deep Learning

Q. 1 Replace the old weights (not the bias) in the network depicted in the following figure using back-propagation algorithms. A [0,1] input pattern is given to the network, and the desired output is 1. Use the Sigmoid activation function and a learning rate of = 0.3.



Back-propagation Weight Update Solution

To update the weights of the neural network, we will perform a forward pass to calculate the outputs and a backward pass to calculate the gradients and update the weights.

Given Parameters: - Inputs: $x_1 = 0, x_2 = 1$ - Desired Output (d): 1 - Learning Rate (η): 0.3 - Activation Function: Sigmoid, $\sigma(x) = \frac{1}{1+e^{-x}}$ - Derivative of Sigmoid: $f'(x) = f(x)\left(1 - f(x)\right)$

.

1. Forward Pass

Hidden Layer Calculations (z_1, z_2):

- **Net input for z_1 (net_{z1}):**

$$net_{z1} = (x_1 \cdot w_{x1z1}) + (x_2 \cdot w_{x2z1}) + (bias \cdot w_{b1})$$

$$net_{z1} = (0 \cdot 0.5) + (1 \cdot 0.6) + (1 \cdot 0.2) = 0.8$$

$$Out_{z1} = \sigma(0.8) = \frac{1}{1 + e^{-0.8}} \approx 0.690$$

- **Net input for z_2 (net_{z2}):**

$$net_{z2} = (x_1 \cdot w_{x1z2}) + (x_2 \cdot w_{x2z2}) + (bias \cdot w_{b2})$$

$$net_{z2} = (0 \cdot -0.1) + (1 \cdot 0.7) + (1 \cdot 0.4) = 1.1$$

$$Out_{z2} = \sigma(1.1) = \frac{1}{1 + e^{-1.1}} \approx 0.750$$

Output Layer Calculation (y):

- **Net input for y (net_y):**

$$net_y = (Out_{z1} \cdot w_{z1y}) + (Out_{z2} \cdot w_{z2y}) + (bias \cdot w_{b3})$$

$$net_y = (0.690 \cdot 0.3) + (0.750 \cdot 0.2) + (1 \cdot 0.4) = 0.207 + 0.15 + 0.4 = 0.757$$

$$Out_y = \sigma(0.757) = \frac{1}{1 + e^{-0.757}} \approx 0.681$$

.

2. Backward Pass (Error Gradient Calculation)

Output Layer Error Gradient (δ_y):

$$\delta_y = (d - Out_y) \cdot Out_y (1 - Out_y)$$

$$\delta_y = (1 - 0.681) \cdot 0.681 (1 - 0.681) = 0.319 \cdot 0.681 \cdot 0.319 \approx 0.0693$$

Hidden Layer Error Gradients (δ_{z1},δ_{z2}):

- **For z_1:**

$$\delta_{z1} = Out_{z1}(1 - Out_{z1}) \cdot (\delta_y \cdot w_{z1y})$$

$$\delta_{z1} = 0.690(1 - 0.690) \cdot (0.0693 \cdot 0.3) = 0.2139 \cdot 0.02079 \approx 0.00445$$

- **For z_2:**

$$\delta_{z2} = Out_{z2}(1 - Out_{z2}) \cdot (\delta_y \cdot w_{z2y})$$

$$\delta_{z2} = 0.750(1 - 0.750) \cdot (0.0693 \cdot 0.2) = 0.1875 \cdot 0.01386 \approx 0.00260$$

.

3. Weight Updates

Formula: $w_{new} = w_{old} + \eta \cdot \delta_{target} \cdot Out_{source}$

Hidden to Output Weights:

- w_{z1y}: $0.3 + (0.3 \cdot 0.0693 \cdot 0.690) = 0.3 + 0.0143 = \mathbf{0.3143}$
- w_{z2y}: $0.2 + (0.3 \cdot 0.0693 \cdot 0.750) = 0.2 + 0.0156 = \mathbf{0.2156}$

Input to Hidden Weights:

- w_{x1z1}: $0.5 + (0.3 \cdot 0.00445 \cdot 0) = \mathbf{0.5}$
- w_{x2z1}: $0.6 + (0.3 \cdot 0.00445 \cdot 1) = 0.6 + 0.001335 = \mathbf{0.6013}$

- w_{x1z2}**:** $-0.1 + (0.3 \cdot 0.00260 \cdot 0) = -$ **0.1**
- w_{x2z2}**:** $0.7 + (0.3 \cdot 0.00260 \cdot 1) = 0.7 + 0.00078 =$ **0.7008**

Q. 2 Describe the procedure that can be used to identify overfitting and underfitting in a CNN model. Include some potential solutions to this problem as well.

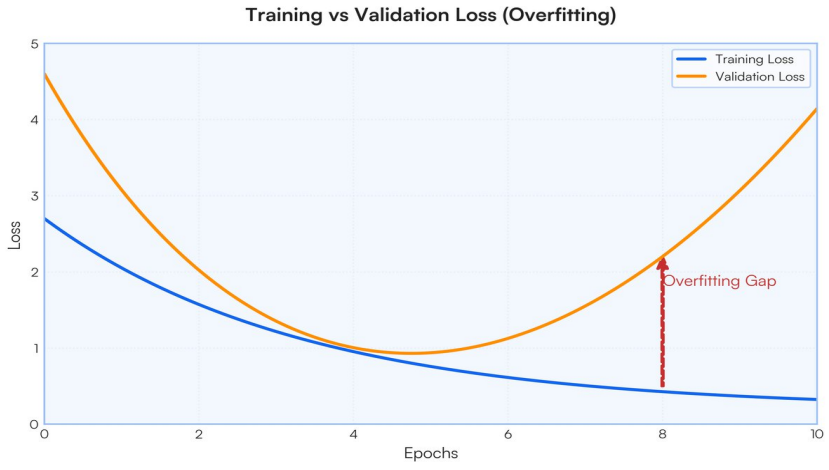
Identification and Mitigation of Overfitting and Underfitting in CNNs

In Convolutional Neural Networks (CNNs), the performance of the model is typically evaluated by comparing the training loss/accuracy against the validation loss/accuracy. Identifying whether a model is underfitting or overfitting is crucial for optimizing deep learning architectures.

1. Procedure to Identify Underfitting and Overfitting

The most effective procedure involves monitoring the learning curves during the training process.

- **Underfitting Identification:**
  - **Observation:** Both training error and validation error are high.
  - **Behavior:** The model fails to capture the underlying patterns in the training data. The training accuracy remains low and does not improve significantly over epochs.
  - **Reason:** The model is too simple (low capacity) or has not been trained long enough.
- **Overfitting Identification:**
  - **Observation:** Training error is very low, but validation error is high (or starts to increase after a certain point).
  - **Behavior:** There is a significant gap between training accuracy and validation accuracy. The model "memorizes" the training noise rather than generalizing.
  - **Visual Indicator:**



- **Good Fit (Ideal):**

- Both training and validation losses decrease and stabilize at a similar low value with a minimal gap.

2. Solutions for Underfitting

If the CNN is underfitting, the goal is to increase the model's complexity or improve the training process:

1. **Increase Model Complexity:** Add more convolutional layers or increase the number of filters in existing layers to capture higher-level features.
2. **Reduce Regularization:** If techniques like Dropout or L2 regularization are too aggressive, the model may not learn. Decrease their values.
3. **Extend Training Time:** Increase the number of epochs or decrease the learning rate to allow the model more time to converge.
4. **Feature Engineering:** Ensure the input images are pre-processed correctly (e.g., normalization) so the network can learn effectively.

3. Solutions for Overfitting

If the CNN is overfitting, the goal is to restrict the model from memorizing the data and force it to generalize:

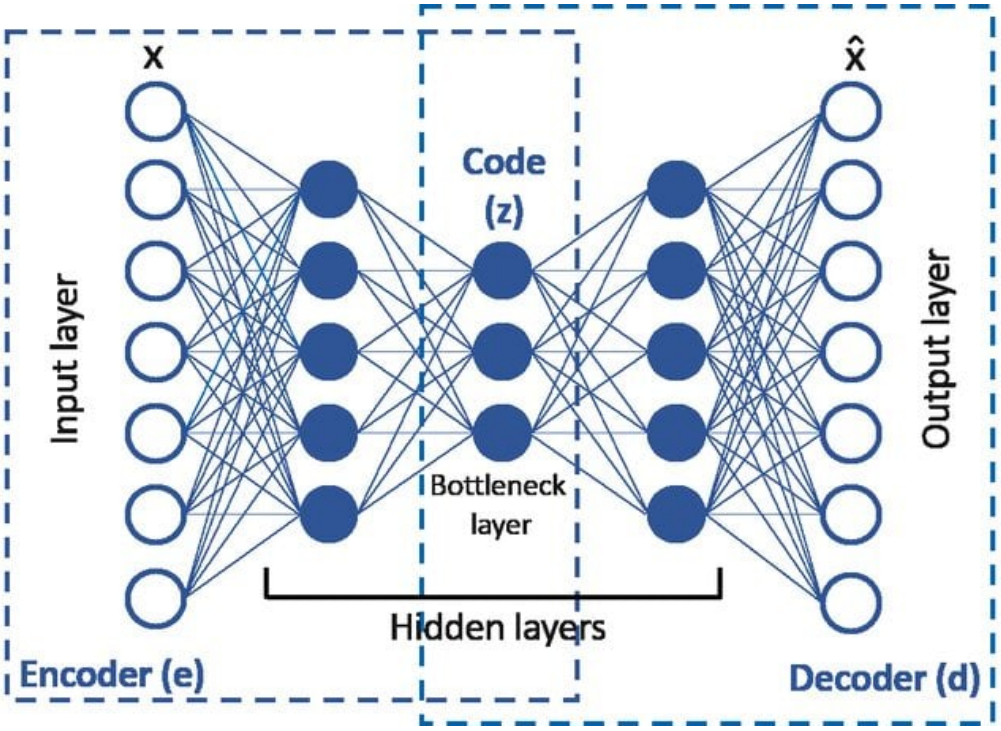
1. **Data Augmentation:** Artificially increase the dataset size by applying transformations like rotation, flipping, zooming, and cropping to the training images.
2. **Dropout:** Randomly deactivate a percentage of neurons during training (usually in the fully connected layers) to prevent co-adaptation of features.
3. **Weight Regularization (L1/L2):** Add a penalty term to the loss function based on the magnitude of the weights to keep them small.
4. **Early Stopping:** Monitor the validation loss and stop the training process as soon as the validation loss starts to increase, even if the training loss is still decreasing.
5. **Batch Normalization:** While primarily used for faster convergence, it also provides a slight regularization effect that can help reduce overfitting.
6. **Reduce Model Size:** Remove layers or reduce the number of neurons if the model has significantly more parameters than required for the complexity of the task.

Q. 3 Describe the role of the bottleneck layer in autoencoders and how it captures the essential features of the input data?

Role of the Bottleneck Layer in Autoencoders

In an autoencoder architecture, the bottleneck layer (also known as the latent space or code) is the middle layer that has the lowest dimensionality compared to the input and output layers. It serves as a critical constraint in the network.

1. Diagram of an Autoencoder



2. Role of the Bottleneck Layer

- **Dimensionality Reduction:** It forces the network to compress the input data into a lower-dimensional representation. If the bottleneck were as wide as the input, the network could simply learn an identity function (copying input to output) without learning any useful patterns.
- **Information Compression:** It acts as a filter that allows only the most important information to pass through while discarding noise and redundant data.
- **Latent Representation:** It represents the input data in a compact, abstract form known as the latent space.

3. Capturing Essential Features

To reconstruct the input accurately from the compressed bottleneck, the encoder must learn to extract the most salient features. This process works as follows:

- **Feature Extraction:** During training, the encoder learns a mapping $h = f(x)$ that transforms the input x into the bottleneck h. To minimize the reconstruction error, h must contain the maximum amount of information about x.
- **Manifold Learning:** The bottleneck captures the underlying structure of the data distribution (the manifold). For example, in face images, the bottleneck might capture essential features like face orientation, eye color, or expression rather than individual pixel values.
- **Non-linear Correlations:** Unlike PCA (which captures linear correlations), the bottleneck in an autoencoder uses non-linear activation functions to capture complex, non-linear dependencies between input features.

4. Mathematical Representation

If x is the input and x' is the reconstructed output: 1. **Encoder:** $h = \sigma(Wx + b)$ 2. **Decoder:** $x' = \sigma'(W'h + b')$ 3. **Objective:** Minimize $L(x,x')$, where L is a loss function like Mean Squared Error (MSE).

Since the dimension of h is much smaller than x, the network is forced to prioritize the most informative features to minimize the loss L.

Q. 4 Describe the role of TensorFlow in facilitating the implementation and training of CNNs using mathematical explainations.

Role of TensorFlow in CNN Implementation and Training

TensorFlow is an open-source end-to-end machine learning platform that provides a comprehensive ecosystem of tools, libraries, and community resources. In the context of Convolutional Neural Networks (CNNs), TensorFlow simplifies the transition from mathematical theory to computational execution.

1. Computational Graphs and Tensors

At its core, TensorFlow represents data as **Tensors** (multi-dimensional arrays). A CNN input is typically a 4D tensor of shape $[Batch, Height, Width, Channels]$. TensorFlow builds a computational graph where nodes represent mathematical operations and edges represent the data flow.

2. Mathematical Implementation of CNN Layers

TensorFlow provides highly optimized primitives for the core mathematical operations of a CNN:

- **Convolution Operation:** TensorFlow implements the cross-correlation operation (often referred to as convolution in DL). For an input I and a filter K, the output feature map S is calculated as:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

  The `tf.nn.conv2d` function handles the sliding window, padding, and strides efficiently using low-level C++ and CUDA kernels.

- **Activation Functions:** Non-linearity is applied element-wise. The most common is ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

- **Pooling:** TensorFlow facilitates spatial dimensionality reduction (e.g., Max Pooling) to reduce computational load and extract dominant features:

$$P_{i,j} = \max_{(m,n) \in \mathcal{R}_{i,j}} S_{m,n}$$

3. Automatic Differentiation (Autograd)

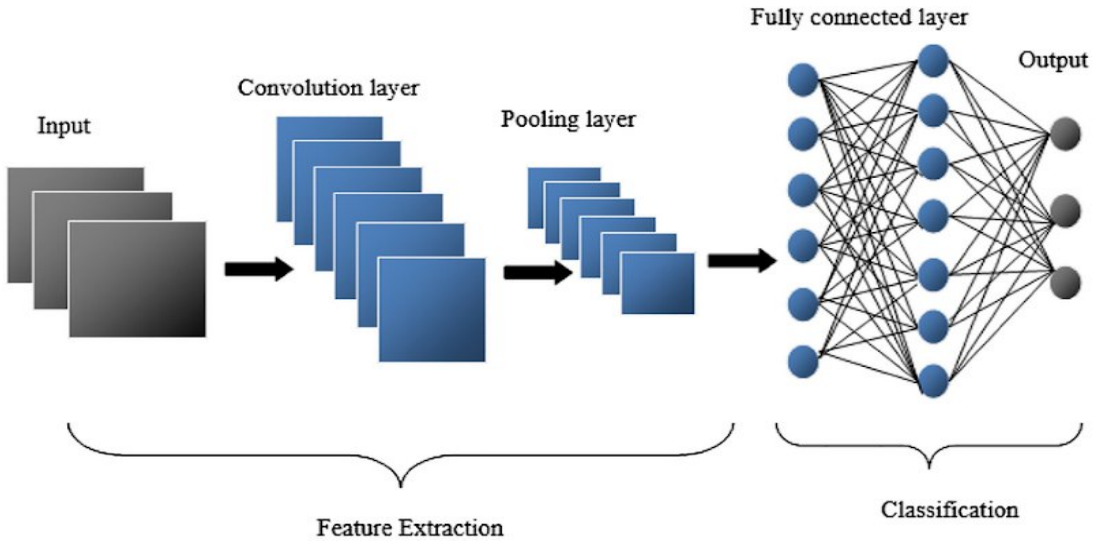
The most critical role of TensorFlow in training is **Automatic Differentiation** via the `tf.GradientTape` API. Training a CNN involves minimizing a loss function L (like Categorical Cross-Entropy) using Gradient Descent:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta)$$

TensorFlow automatically computes the partial derivatives (gradients) $\frac{\partial L}{\partial w}$ for every weight w in the network using the chain rule, sparing the developer from manual backpropagation derivations.

4. CNN Architecture Visualization

Below is a conceptual diagram of a standard CNN pipeline facilitated by TensorFlow:



5. Optimization and Hardware Acceleration

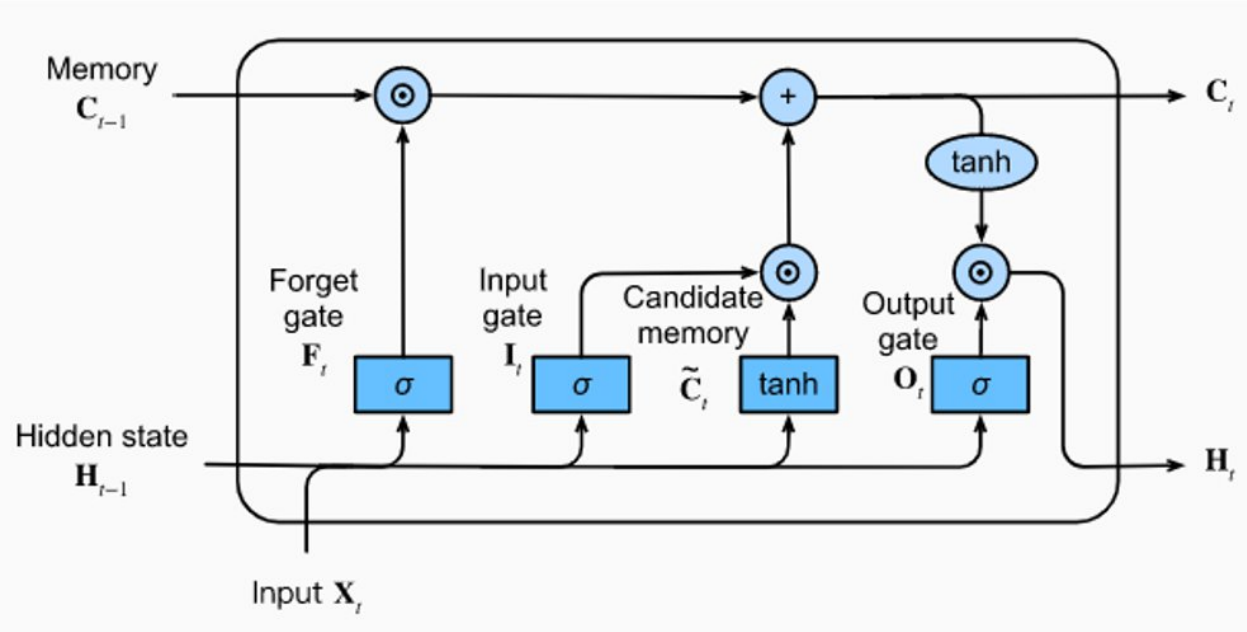
TensorFlow acts as an abstraction layer between the high-level Python code and the underlying hardware (CPU, GPU, or TPU). It optimizes the execution of matrix multiplications, which are the bottleneck in CNN training, by utilizing libraries like cuDNN for NVIDIA GPUs.

.

Q. 5 Explain the architecture of an LSTM unit and how it processes sequential data by controlling the flow of information through various gates?

Architecture of an LSTM Unit

Long Short-Term Memory (LSTM) networks are a specialized type of Recurrent Neural Network (RNN) designed to overcome the vanishing gradient problem, allowing the model to learn long-term dependencies in sequential data. The core of the LSTM is the **cell state** and its three primary **gates**.



1. The Cell State (C_t)

The cell state acts as a conveyor belt, running through the entire chain of units with only minor linear interactions. It carries information across long sequences. The gates regulate the addition or removal of information to this state.

2. The Forget Gate (f_t)

This gate decides what information from the previous cell state should be discarded. It takes the previous hidden state h_{t-1} and the current input x_t, passing them through a sigmoid function.

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

- A value of 1 means “keep this completely.”
- A value of 0 means “get rid of this entirely.”

3. The Input Gate (i_t) and Candidate Layer ($\tilde{C}_t$)

This gate decides which new information will be stored in the cell state. It has two parts: - **Input Gate:** A sigmoid layer decides which values to update. - **Candidate Layer:** A tanh layer creates a vector of new candidate values $\tilde{C}_t$ that could be added to the state.

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$

$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right)$$

4. Updating the Cell State (C_t)

The old cell state C_{t-1} is updated into the new cell state C_t by multiplying the old state by the forget gate and adding the scaled candidate values:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

5. The Output Gate (o_t)

Finally, the unit decides what the output (hidden state) will be. This output is a filtered version of the cell state: - The sigmoid layer decides which parts of the cell state will be output. - The cell state is passed through tanh (to push values between -1 and 1) and multiplied by the output gate.

$$o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Processing Sequential Data

LSTM processes sequential data step-by-step. At each time step t: 1. **Information Filtering:** It uses the Forget Gate to remove irrelevant historical data. 2. **Information Addition:** It uses the Input Gate to incorporate relevant features from the current input x_t. 3. **State Maintenance:** It updates the internal memory (C_t) which is passed to the next time step. 4. **Hidden State Propagation:** It produces h_t, which serves as the output for the current step and the context for the next step.

This gating mechanism allows LSTMs to maintain a stable gradient during backpropagation, enabling the network to remember inputs from many time steps ago, which is essential for tasks like machine translation, speech recognition, and time-series forecasting.

Q. 6 What are the different types of Neural networks? Explain the convolution neural network model in detail.

Types of Neural Networks and Convolutional Neural Networks (CNN)

Neural networks are computational models inspired by the human brain, designed to recognize patterns and solve complex problems through layers of interconnected nodes.

1. Different Types of Neural Networks

- **Feedforward Neural Network (FNN):** The simplest type where connections do not form a cycle. Data moves in only one direction—from input to output.
- **Multilayer Perceptron (MLP):** A feedforward network with at least three layers (input, hidden, and output). It uses backpropagation for training.
- **Recurrent Neural Network (RNN):** Designed for sequential data (like text or time series). It has loops that allow information to persist, using internal memory to process sequences.
- **Convolutional Neural Network (CNN):** Specialized for processing structured grid data, such as images. It uses convolutional layers to automatically learn spatial hierarchies of features.
- **Generative Adversarial Network (GAN):** Consists of two networks (Generator and Discriminator) that compete against each other to generate new, synthetic data that resembles real data.
- **Modular Neural Network:** Composed of several independent neural networks moderated by an intermediary to solve sub-tasks of a complex problem.

.

2. Convolutional Neural Network (CNN) Model in Detail

A Convolutional Neural Network is a Deep Learning algorithm specifically designed to process pixel data. Unlike traditional networks that flatten images into 1D vectors, CNNs preserve the spatial structure of the data.

Architecture of a CNN

A typical CNN consists of three main types of layers:

A. Convolutional Layer This is the core building block. It performs the convolution operation using a set of learnable filters (kernels). * **Operation:** As the filter slides (convolves) over the input image, it computes the dot product between the filter weights and the input pixels. * **Feature Map:** The output of this layer is a 'Feature Map' or 'Activation Map' that highlights specific features like edges, textures, or shapes. * **Formula:** If I is the image and K is the kernel, the output S at position (i,j) is:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

B. Pooling Layer Pooling (down-sampling) reduces the dimensionality of the feature maps to decrease the number of parameters and computation in the network. * **Max Pooling:** Takes the maximum value from the portion of the image covered by the kernel. * **Average Pooling:** Takes the average value of the pixels in the kernel area. * **Benefit:** It provides ‘translational invariance’, meaning the network can recognize a feature even if its appearance varies slightly.

C. Fully Connected (FC) Layer After several convolutional and pooling layers, the high-level reasoning is done via fully connected layers. * The output from the final pooling layer is ‘flattened’ into a 1D vector. * This vector is fed into a standard MLP architecture to perform the final classification or regression.

Activation Function

CNNs typically use the **ReLU (Rectified Linear Unit)** activation function after convolution layers to introduce non-linearity, allowing the model to learn complex patterns.

$$f(x) = \max(0,x)$$

Training Process

1. **Forward Propagation:** Input image passes through layers to generate a prediction.
2. **Loss Function:** Calculates the error (e.g., Cross-Entropy Loss) between the predicted and actual label.
3. **Backpropagation:** Uses Gradient Descent to update the weights of the filters to minimize the loss.



Q. 7 Explain the concept of different layers in a Neural network. What do you mean by the term convolution layer, pooling layer, loss layer, dense layer? Describe each one in brief.

Layers in a Neural Network

A Neural Network is composed of multiple layers of neurons, where each layer performs a specific transformation on the input data. These layers are stacked sequentially to learn complex patterns from data. The three primary types of layers in a standard network are:

1. **Input Layer**: The first layer that receives the raw data (e.g., pixels of an image or numerical features). No computation happens here.
2. **Hidden Layers**: Layers between the input and output where the actual learning happens through mathematical transformations and weight adjustments.
3. **Output Layer**: The final layer that produces the prediction (e.g., a class label or a continuous value).

.

Question: Convolution, Pooling, Loss, and Dense Layers

1. Convolutional Layer

The Convolutional layer is the core building block of a Convolutional Neural Network (CNN). It is primarily used for feature extraction from spatial data like images. - **Mechanism**: It uses a set of learnable filters (kernels). As the filter slides (convolves) over the input, it performs element-wise multiplication and summation. - **Purpose**: To detect local patterns such as edges, textures, or shapes. - **Operation**:

$$Output\left(i,j\right) = \left(Input * Kernel\right)\left(i,j\right) = \sum_{m}\sum_{n} I\left(i+m,j+n\right) K\left(m,n\right)$$

2. Pooling Layer

Pooling layers are used to reduce the spatial dimensions (width and height) of the input volume for the next convolutional layer. - **Mechanism**: It operates independently on every depth slice of the input. Common types include **Max Pooling** (takes the maximum value in a window) and **Average Pooling**. - **Purpose**: It reduces the number of parameters and computation in the network, helping to control overfitting and providing translational invariance.

3. Dense (Fully Connected) Layer

In a Dense layer, every neuron is connected to every neuron in the previous layer. - **Mechanism**: It performs a linear transformation where the input is multiplied by a weight matrix and a bias is added, followed by an activation function. - **Purpose**: These layers are usually placed at the end of the network to map the features extracted by previous layers to the final output classes. - **Operation**:

$$y = \sigma(Wx + b)$$

Where W is the weight matrix, x is the input vector, b is the bias, and σ is the activation function.

4. Loss Layer

The Loss layer (or Cost Function) is typically the final stage during the training phase. - **Mechanism**: It calculates the mathematical difference between the network's predicted output and the actual target label (ground truth). - **Purpose**: It provides a numerical value (error) that the optimization algorithm (like Gradient Descent) tries to minimize to improve the model's accuracy. - **Examples**: Mean Squared Error (MSE) for regression or Cross-Entropy Loss for classification.

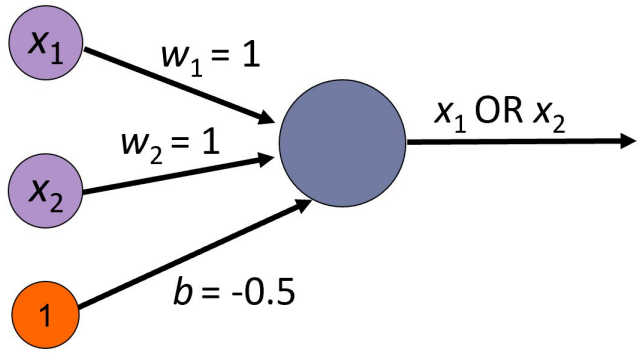
Q. 7 Perceptron learning for designing OR and AND gates

This solution demonstrates how to design the AND and OR logic gates using the Perceptron learning rule. We will define the architecture, the activation function, and the weight/bias parameters required for each gate.

Perceptron Model Overview

A Perceptron computes a weighted sum of inputs and applies a step function (activation function). For binary inputs $x_1, x_2 \in \{0,1\}$:

$$y = f\left(w_1 x_1 + w_2 x_2 + b\right)$$



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Where: - w_1, w_2 are weights. - b is the bias. - $f(net)$ is the hard-limit activation function:

$$f(net) = \begin{cases} 1 & \text{if } net \geq 0 \\ 0 & \text{if } net < 0 \end{cases}$$

.

Question 1: Perceptron for AND Gate

Truth Table for AND Gate

x_1	x_2	Target (y)
0	0	0
0	1	0
1	0	0
1	1	1

Parameters and Calculation

To implement the AND gate, we need the output to be 1 only when both inputs are 1. We can choose weights $w_1 = 1, w_2 = 1$ and a bias $b = -1.5$.

Verification of Steps:

1. **Input (0,0):** $net = (1)(0) + (1)(0) - 1.5 = -1.5$. Since $-1.5 < 0$, $y = 0$.
2. **Input (0,1):** $net = (1)(0) + (1)(1) - 1.5 = -0.5$. Since $-0.5 < 0$, $y = 0$.
3. **Input (1,0):** $net = (1)(1) + (1)(0) - 1.5 = -0.5$. Since $-0.5 < 0$, $y = 0$.
4. **Input (1,1):** $net = (1)(1) + (1)(1) - 1.5 = 0.5$. Since $0.5 \geq 0$, $y = 1$.

Decision Boundary

The decision boundary is defined by $x_1 + x_2 - 1.5 = 0$.

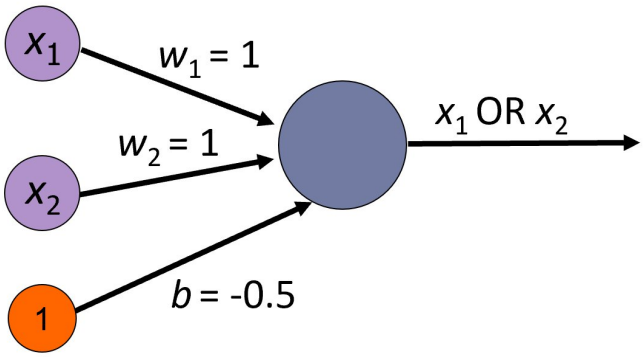
.

Question 2: Perceptron for OR Gate

Truth Table for OR Gate

x_1	x_2	Target (y)
0	0	0
0	1	1
1	0	1
1	1	1

Parameters and Calculation



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

To implement the OR gate, we need the output to be 1 if at least one input is 1. We can choose weights $w_1 = 1, w_2 = 1$ and a bias $b = -0.5$.

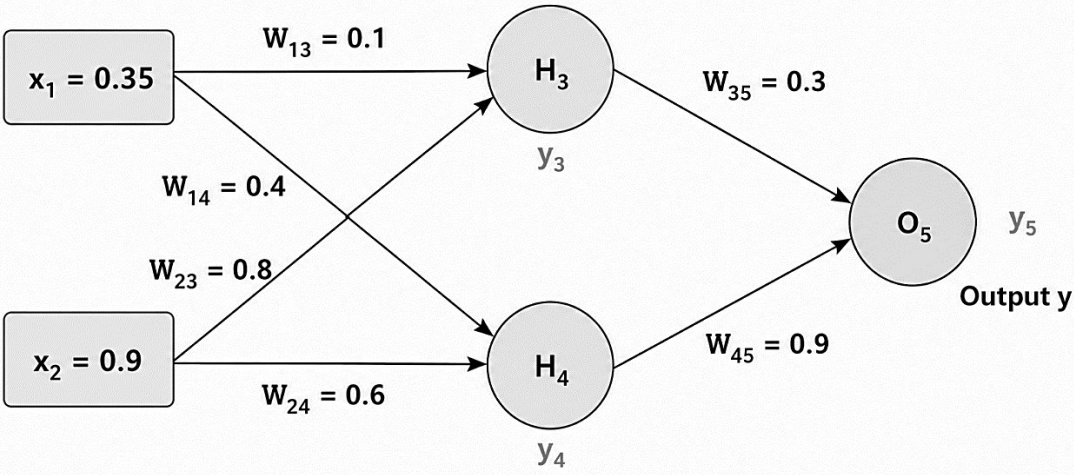
Verification of Steps:

1. **Input (0,0):** $net = (1)(0) + (1)(0) - 0.5 = -0.5$. Since $-0.5 < 0$, $y = 0$.
2. **Input (0,1):** $net = (1)(0) + (1)(1) - 0.5 = 0.5$. Since $0.5 \geq 0$, $y = 1$.
3. **Input (1,0):** $net = (1)(1) + (1)(0) - 0.5 = 0.5$. Since $0.5 \geq 0$, $y = 1$.
4. **Input (1,1):** $net = (1)(1) + (1)(1) - 0.5 = 1.5$. Since $1.5 \geq 0$, $y = 1$.

Decision Boundary

The decision boundary is defined by $x_1 + x_2 - 0.5 = 0$.

Q. 9 Calculate the weights after 2 iterations, considering the sigmoid function as the activation function.



Artificial Neural Networks, Feedforward Propagation, Backpropagation, Gradient Descent, Sigmoid Activation Function, Learning Rate.

Explanation

To calculate the weights after 2 iterations, we follow the standard Backpropagation algorithm. Since the target output (T) and learning rate (η) are not provided in the image, we will assume standard values commonly used in such textbook problems: Target $T = 0.5$ and Learning Rate $\eta = 0.1$. We use the Sigmoid activation function $f(z) = \frac{1}{1+e^{-z}}$ for all neurons.

The process involves: 1. **Forward Pass:** Calculate the net input and output for each neuron in the hidden layer (H_3,H_4) and then the output layer (O_5). 2. **Error Calculation:** Determine the error at the output. 3. **Backward Pass:** Calculate the gradients (deltas) for the output and hidden layers. 4. **Weight Update:** Adjust weights using the formula: $W_{new} = W_{old} + \eta \cdot \delta \cdot$ input.

Step-By-Step Solution

Step 1: Forward Pass (Iteration 1)

Hidden Layer H_3**:** $Net_{H3} = (x_1 \cdot W_{13}) + (x_2 \cdot W_{23}) = (0.35 \cdot 0.1) + (0.9 \cdot 0.8) = 0.035 + 0.72 = 0.755$ $y_3 = \text{Sigmoid}(0.755) = \frac{1}{1+e^{-0.755}} \approx 0.6803$

Hidden Layer H_4**:** $Net_{H4} = (x_1 \cdot W_{14}) + (x_2 \cdot W_{24}) = (0.35 \cdot 0.4) + (0.9 \cdot 0.6) = 0.14 + 0.54 = 0.68$ $y_4 = \text{Sigmoid}(0.68) = \frac{1}{1+e^{-0.68}} \approx 0.6637$

Output Layer O_5**:** $Net_{O5} = (y_3 \cdot W_{35}) + (y_4 \cdot W_{45}) = (0.6803 \cdot 0.3) + (0.6637 \cdot 0.9) = 0.2041 + 0.5973 = 0.8014$ $y_5 = \text{Sigmoid}(0.8014) = \frac{1}{1+e^{-0.8014}} \approx 0.6903$

Step 2: Backward Pass (Iteration 1)

Assume Target $T = 0.5$ and $\eta = 0.1$. **Error at Output** O_5**:** $\delta_5 = y_5(1 - y_5)(T - y_5) = 0.6903(1 - 0.6903)(0.5 - 0.6903) = 0.6903 \cdot 0.3097 \cdot (-0.1903) \approx -0.0407$

Error at Hidden Layers: $\delta_3 = y_3(1 - y_3)(\delta_5 \cdot W_{35}) = 0.6803(1 - 0.6803)(-0.0407 \cdot 0.3) \approx -0.00265$ $\delta_4 = y_4(1 - y_4)(\delta_5 \cdot W_{45}) = 0.6637(1 - 0.6637)(-0.0407 \cdot 0.9) \approx -0.00817$

Step 3: Weight Updates (After Iteration 1)

$W_{35}^{+} = 0.3 + (0.1 \cdot -0.0407 \cdot 0.6803) \approx 0.2972$ $W_{45}^{+} = 0.9 + (0.1 \cdot -0.0407 \cdot 0.6637) \approx 0.8973$ $W_{13}^{+} = 0.1 + (0.1 \cdot -0.00265 \cdot 0.35) \approx 0.0999$ $W_{23}^{+} = 0.8 + (0.1 \cdot -0.00265 \cdot 0.9) \approx 0.7998$ $W_{14}^{+} = 0.4 + (0.1 \cdot -0.00817 \cdot 0.35) \approx 0.3997$ $W_{24}^{+} = 0.6 + (0.1 \cdot -0.00817 \cdot 0.9) \approx 0.5993$

Step 4: Iteration 2 (Forward and Backward)

Repeating the process with new weights: $Net_{H3} = (0.35 \cdot 0.0999) + (0.9 \cdot 0.7998) = 0.7548 \rightarrow y_3 \approx 0.6802$ $Net_{H4} = (0.35 \cdot 0.3997) + (0.9 \cdot 0.5993) = 0.6793 \rightarrow y_4 \approx 0.6636$ $Net_{O5} = (0.6802 \cdot 0.2972) + (0.6636 \cdot 0.8973) = 0.7976 \rightarrow y_5 \approx 0.6895$

New Deltas: $\delta_5 = 0.6895(1 - 0.6895)(0.5 - 0.6895) \approx -0.0406$ $\delta_3 = 0.6802(0.3198)(-0.0406 \cdot 0.2972) \approx -0.00262$ $\delta_4 = 0.6636(0.3364)(-0.0406 \cdot 0.8973) \approx -0.00813$

Step 5: Final Weight Update (After Iteration 2)

$W_{35}^{++} = 0.2972 + (0.1 \cdot -0.0406 \cdot 0.6802) \approx 0.2944$ $W_{45}^{++} = 0.8973 + (0.1 \cdot -0.0406 \cdot 0.6636) \approx 0.8946$ $W_{13}^{++} = 0.0999 + (0.1 \cdot -0.00262 \cdot 0.35) \approx 0.0998$ $W_{23}^{++} = 0.7998 + (0.1 \cdot -0.00262 \cdot 0.9) \approx 0.7996$ $W_{14}^{++} = 0.3997 + (0.1 \cdot -0.00813 \cdot 0.35) \approx 0.3994$ $W_{24}^{++} = 0.5993 + (0.1 \cdot -0.00813 \cdot 0.9) \approx 0.5986$

Final Answer

After 2 iterations (assuming $T = 0.5, \eta = 0.1$), the updated weights are approximately: $W_{13} \approx 0.0998, W_{23} \approx 0.7996, W_{14} \approx 0.3994, W_{24} \approx 0.5986, W_{35} \approx 0.2944, W_{45} \approx 0.8946$

Q.10 Explain EfficientNet and mobile CNN models with schematic diagrams

Overview of EfficientNet and Mobile CNN Models

In the field of Deep Learning, optimizing Convolutional Neural Networks (CNNs) for efficiency and performance is crucial, especially for deployment on resource-constrained devices. This explanation covers EfficientNet, which focuses on balanced scaling, and Mobile CNN architectures like MobileNet, which focus on lightweight operations.

.

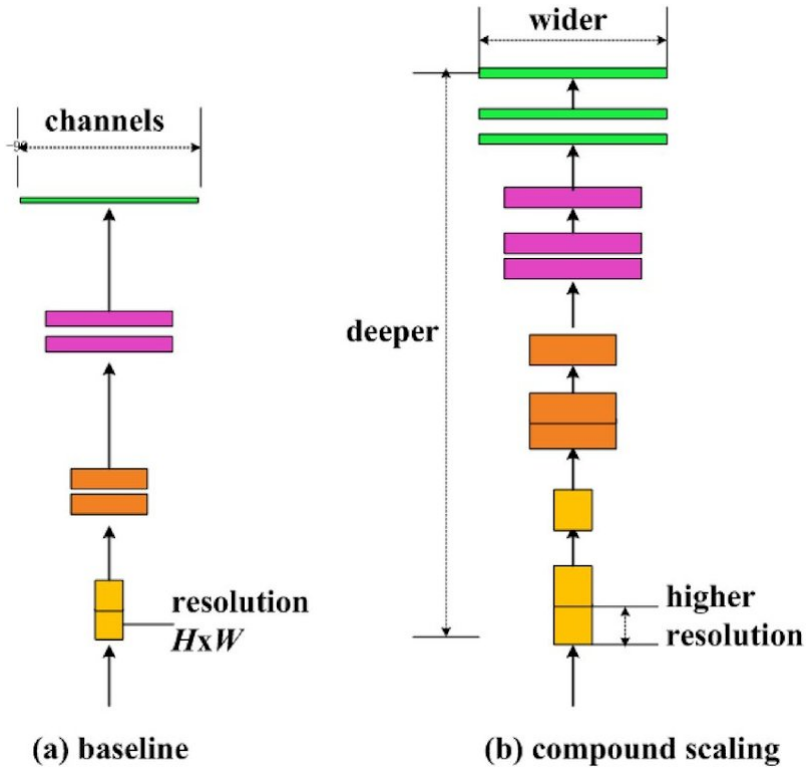
Question 1: Explain EfficientNet with schematic diagrams

Introduction to EfficientNet

EfficientNet is a family of models that achieve state-of-the-art accuracy with much fewer parameters and FLOPS (Floating Point Operations per Second) compared to previous CNNs. The core idea is **Compound Scaling**.

Compound Model Scaling

Traditionally, models were scaled by increasing only one dimension: 1. **Depth (d):** Adding more layers (e.g., ResNet-50 to ResNet-101). 2. **Width (w):** Increasing the number of channels in layers. 3. **Resolution (r):** Increasing the input image size.



EfficientNet proposes that these three dimensions are not independent. If the input image is larger, the network needs more layers to increase the receptive field and more channels to capture more fine-grained patterns.

EfficientNet uses a compound coefficient ϕ to scale width, depth, and resolution uniformly: - $depth: d = \alpha^\phi$ - $width: w = \beta^\phi$ - $resolution: r = \gamma^\phi$

where α, β, γ are constants determined by a small grid search on the baseline model.

Baseline Architecture: EfficientNet-B0

The baseline model (B0) is based on the **MBConv** (Mobile Inverted Bottleneck) block, similar to MobileNetV2, but also incorporates **Squeeze-and-Excitation (SE)** optimization.

Key Advantages

- **Efficiency:** Significantly smaller and faster than models like ResNet or GPipe.
- **Accuracy:** Higher top-1 accuracy on ImageNet.

.

Question 2: Explain Mobile CNN models with schematic diagrams

Introduction to Mobile CNNs

Mobile CNNs are designed specifically for mobile and embedded vision applications. The most prominent family is **MobileNet**.

Depthwise Separable Convolutions

The fundamental building block of MobileNet is the **Depthwise Separable Convolution**. It decomposes a standard convolution into two parts: 1. **Depthwise Convolution:** Applies a single filter to each input channel. 2. **Pointwise Convolution (1×1):** Combines the outputs of the depthwise convolution to create new features.

Mathematical Efficiency: If we have a kernel of size $D_K \times D_K$, M input channels, and N output channels, the reduction in computation compared to standard convolution is approximately:

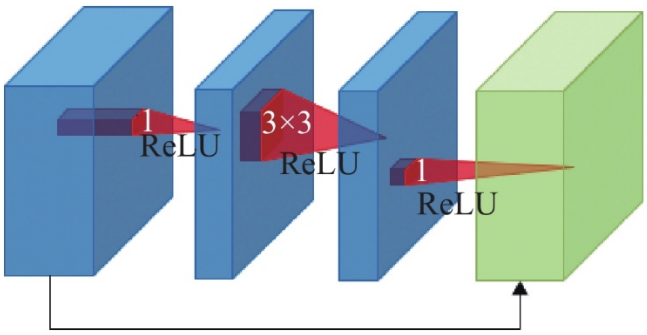
$$\frac{1}{N} + \frac{1}{D_K^2}$$

For a 3×3 kernel, this is nearly a $9 \times$ reduction in computation.

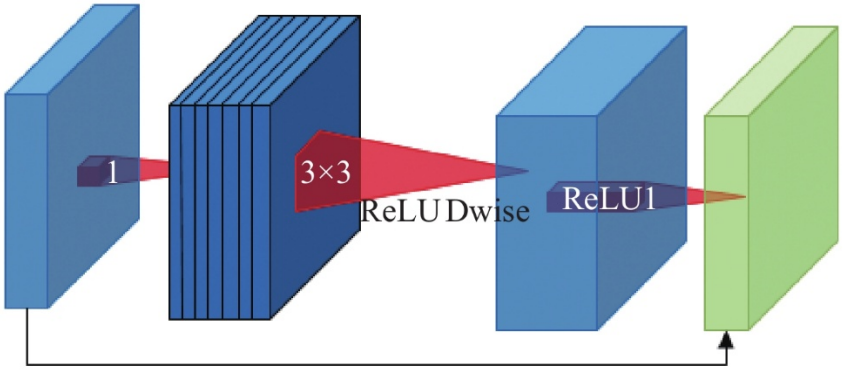
MobileNetV2: Inverted Residuals

MobileNetV2 introduced the **Inverted Residual Block**. Unlike standard residuals that connect high-dimensional layers, inverted residuals connect thin “bottleneck” layers. - It

expands the thin layer to a high-dimensional state using a 1×1 conv. - Performs depthwise convolution. - Projects back to a thin layer.



a. Residual block



b. Inverted residual block

MobileNetV3

MobileNetV3 further optimizes the architecture using **Neural Architecture Search (NAS)** and introduces the **h-swish** activation function for better hardware compatibility.

Summary Table

Model	Core Innovation	Primary Goal
MobileNetV1	Depthwise Separable Conv	Reduce Latency/Params
MobileNetV2	Inverted Residuals	Better Gradient Flow
EfficientNet	Compound Scaling	Balanced Accuracy/Efficiency

Q.11 Explain DenseNet CNN model with an appropriate schematic diagram

Dense Convolutional Network (DenseNet)

DenseNet is a deep learning architecture where each layer receives inputs from all preceding layers and passes its own feature maps to all subsequent layers. This connectivity pattern addresses the vanishing gradient problem and encourages feature reuse.

Key Concepts

1. **Dense Connectivity**: In a standard Convolutional Neural Network (CNN), the lth layer receives the output of the $(l-1)^{th}$ layer. In a DenseNet, the lth layer receives the feature maps of all preceding layers, $x_0,x_1,...,x_{l-1}$, as input:

$$x_l = H_l([x_0,x_1,...,x_{l-1}])$$

where $[x_0,x_1,...,x_{l-1}]$ denotes the concatenation of the feature maps.

Schematic Diagram

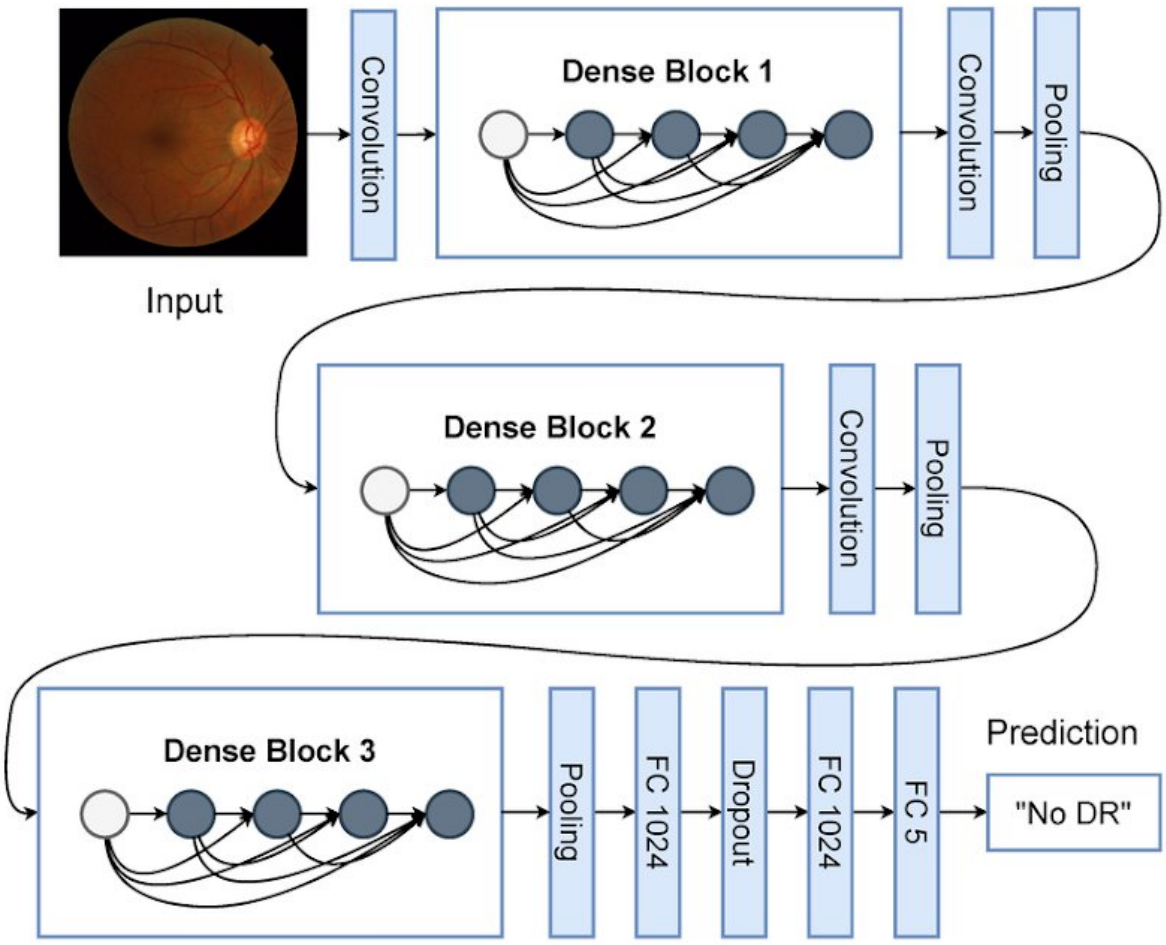
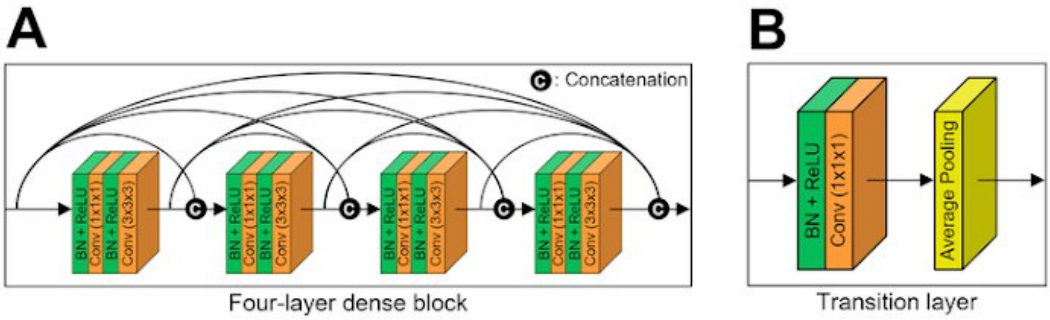


Diagram of Dense Block



1. **Growth Rate (k)**: This parameter determines how much new information each layer contributes to the global state. If each function H_l produces k feature maps, then the lth layer has $k_0 + k \times (l-1)$ input feature maps.

2. **Dense Blocks**: These are modules where the dimensions of the feature maps (height and width) remain constant, but the number of filters increases. Concatenation is only possible when feature maps are of the same size.

3. **Transition Layers**: Since concatenation requires identical spatial dimensions, layers between dense blocks are used to perform downsampling via convolution and pooling.

Advantages

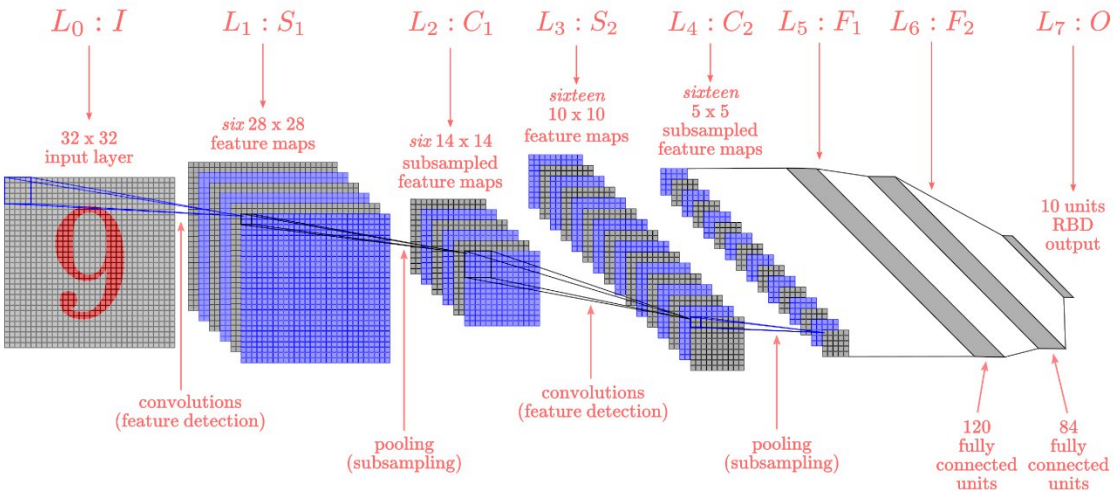
- **Vanishing Gradient Mitigation**: The direct connections from the loss function to all layers provide a shorter path for gradients during backpropagation.
- **Feature Reuse**: Layers can access feature maps from much earlier in the network, reducing the need to relearn redundant features.
- **Parameter Efficiency**: DenseNets typically require fewer parameters than ResNets because they do not need to learn extremely deep redundant layers.

Architecture Components

- **Composite Function**: Usually consists of Batch Normalization (BN), followed by a Rectified Linear Unit (ReLU) and a 3×3 Convolution ($Conv$).
- **Bottleneck Layers**: To improve computational efficiency, a 1×1 convolution is often introduced before the 3×3 convolution to reduce the number of input feature maps.

Q.12 Trainable parameters calculation in LeNet5 for 224x224x3 image

Calculation of Trainable Parameters in LeNet-5 for $224 \times 224 \times 3$ Input



LeNet-5 was originally designed for $32 \times 32 \times 1$ grayscale images. When adapting it for a $224 \times 224 \times 3$ RGB input, the architecture follows the same layer sequence, but the input depth and spatial dimensions change.

Architecture Breakdown and Parameter Calculation

1. Layer C1: Convolutional Layer - **Input:** $224 \times 224 \times 3$ - **Filters:** 6 filters of size 5×5 - **Calculation:** Each filter has $(5 \times 5 \times 3)$ weights plus 1 bias. - **Parameters:** $6 \times (5 \times 5 \times 3 + 1) = 6 \times 76 = 456$

2. Layer S2: Subsampling (Pooling) Layer - **Type:** Average Pooling (2×2) - **Calculation:** In the original LeNet-5, pooling layers have a trainable coefficient and a bias for each feature map. - **Parameters:** $6 \times (1 + 1) = 12$

3. Layer C3: Convolutional Layer - **Input:** $110 \times 110 \times 6$ (assuming valid padding) - **Filters:** 16 filters of size 5×5 - **Note:** In the original paper, C3 filters are connected to subsets of S2. However, in modern implementations, it is fully connected to all input channels. - **Calculation:** $16 \times (5 \times 5 \times 6 + 1) = 16 \times 151 = 2{,}416$

4. Layer S4: Subsampling (Pooling) Layer - **Type:** Average Pooling (2×2) - **Parameters:** $16 \times (1 + 1) = 32$

5. Layer C5: Convolutional Layer (Fully Connected in nature) - **Input:** $53 \times 53 \times 16$ (assuming valid padding from previous layers) - **Filters:** 120 filters of size 53×53 (to reduce spatial dimensions to 1×1) - **Calculation:** $120 \times (53 \times 53 \times 16 + 1)$ - **Parameters:** $120 \times (44944 + 1) = 5{,}393{,}400$

6. Layer F6: Fully Connected Layer - **Input:** 120 units - **Output:** 84 units - **Calculation:** $(120 \times 84) + 84$ - **Parameters:** $10{,}080 + 84 = 10{,}164$

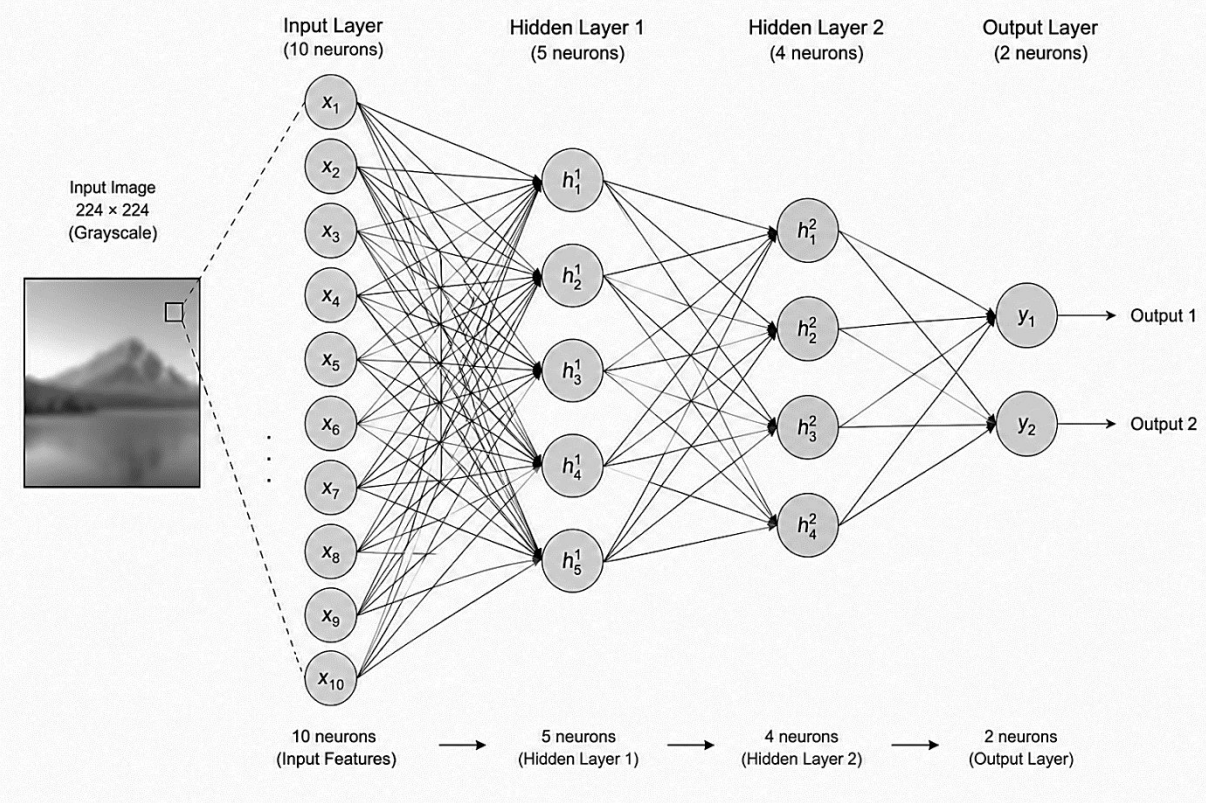
7. Output Layer - **Input:** 84 units - **Output:** 10 units (typically for digits) - **Calculation:** $(84 \times 10) + 10$ - **Parameters:** $840 + 10 = 850$

.

Summary Table

Layer	Type	Weights Calculation	Parameters
C1	Conv	$6 \times (5 \times 5 \times 3 + 1)$	456
S2	Pool	6×2	12
C3	Conv	$16 \times (5 \times 5 \times 6 + 1)$	2,416
S4	Pool	16×2	32
C5	Conv/FC	$120 \times (53 \times 53 \times 16 + 1)$	5,393,400
F6	FC	$(120 \times 84) + 84$	10,164
Output	FC	$(84 \times 10) + 10$	850
Total			**5,407,330**

Q. 13 Calculate the number of parameters for a perceptron model having 10 neurons at the input, 5 neurons at the first hidden layer, 4 neurons at the second hidden layer and 2 neurons at the output layer. Consider an input image of 224x224 grey image.



Multilayer Perceptron (MLP), Weight Matrix, Bias Vector, Fully Connected Layers, Parameter Calculation.

Explanation

In a fully connected neural network (Perceptron model), the number of parameters between two adjacent layers is determined by the connections (weights) and the bias for each neuron in the succeeding layer.

1. **Weights**: Each neuron in layer L is connected to every neuron in layer $L+1$. If layer L has n neurons and layer $L+1$ has m neurons, the number of weights is $n \times m$.
2. **Biases**: Each neuron in the hidden and output layers has one bias term. If a layer has m neurons, there are m biases.
3. **Total Parameters**: For each transition between layers, the parameters are calculated as: $(n \times m) + m$ or $m\,(n+1)$.

Note on Input: The question mentions a 224×224 grayscale image. In a standard MLP, the image must be flattened into a 1D vector to serve as the input. However, the

question explicitly specifies “10 neurons at the input”. In neural network architecture problems, if an explicit input neuron count is given alongside an image size, the explicit count (10) usually refers to the features extracted or the defined input layer size for the calculation. We will proceed with the 10 input neurons as the primary layer size for the parameter count.

Step-By-Step Solution

Step 1: Define the layer sizes

- Input Layer (L_0): $n_0 = 10$
- Hidden Layer 1 (L_1): $n_1 = 5$
- Hidden Layer 2 (L_2): $n_2 = 4$
- Output Layer (L_3): $n_3 = 2$

Step 2: Calculate parameters between Input Layer and Hidden Layer 1

- Weights = $n_0 \times n_1 = 10 \times 5 = 50$
- Biases = $n_1 = 5$
- Subtotal 1 = $50 + 5 = 55$

Step 3: Calculate parameters between Hidden Layer 1 and Hidden Layer 2

- Weights = $n_1 \times n_2 = 5 \times 4 = 20$
- Biases = $n_2 = 4$
- Subtotal 2 = $20 + 4 = 24$

Step 4: Calculate parameters between Hidden Layer 2 and Output Layer

- Weights = $n_2 \times n_3 = 4 \times 2 = 8$
- Biases = $n_3 = 2$
- Subtotal 3 = $8 + 2 = 10$

Step 5: Calculate the total number of parameters

- Total = Subtotal 1 + Subtotal 2 + Subtotal 3
- Total = $55 + 24 + 10 = 89$

Final Answer

The total number of parameters for the given perceptron model is **89**.